```
hart 1 starting
hart 2 starting
init: starting sh
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
forkfork: ok
ALL COW TESTS PASSED
$ usertests -q
usertests starting
test copyin: OK
test copyout: OK
test copyinstr1: OK
test copyinstr2: OK
test copyinstr3: OK
test rwsbrk: OK
test truncate1: OK
test truncate2: OK
test truncate3: OK
test openiput: OK
test exitiput: OK
test iput: OK
test opentest: OK
test writetest: OK
test writebig: OK
test createtest: OK
test dirtest: OK
test exectest: OK
test pipe1: OK
test killstatus: OK
test preempt: kill... wait... OK
test exitwait: OK
test reparent: OK
test twochildren: OK
test forkfork: OK
test forkforkfork: OK
test reparent2: OK
test mem: OK
test sharedfd: OK
test fourfiles: OK
test createdelete: OK
test unlinkread: OK
test linktest: OK
test concreate: OK
test linkunlink: OK
test subdir: OK
test bigwrite: OK
test bigfile: OK
test fourteen: OK
test rmdot: OK
```

```
test rmdot: OK
test dirfile: OK
test iref: OK
test forktest: OK
test sbrkbasic: OK
test sbrkmuch: OK
usertrap(): unexpected scause 0xf pid=7829
            sepc=0x4442 stval=0x26013000
usertrap(): unexpected scause 0xf pid=7830
            sepc=0x4442 stval=0x27013000
usertrap(): unexpected scause 0xf pid=7831
            sepc=0x4442 stval=0x28013000
usertrap(): unexpected scause 0xf pid=7832
            sepc=0x4442 stval=0x29013000
usertrap(): unexpected scause 0xf pid=7833
            sepc=0x4442 stval=0x2a013000
usertrap(): unexpected scause 0xf pid=7834
            sepc=0x4442 stval=0x2b013000
usertrap(): unexpected scause 0xf pid=7835
            sepc=0x4442 stval=0x2c013000
usertrap(): unexpected scause 0xf pid=7836
            sepc=0x4442 stval=0x2d013000
usertrap(): unexpected scause 0xf pid=7837
            sepc=0x4442 stval=0x2e013000
usertrap(): unexpected scause 0xf pid=7838
            sepc=0x4442 stval=0x2f013000
usertrap(): unexpected scause 0xf pid=7839
            sepc=0x4442 stval=0x30013000
usertrap(): unexpected scause 0xf pid=7840
            sepc=0x4442 stval=0x31013000
usertrap(): unexpected scause 0xf pid=7841
            sepc=0x4442 stval=0x32013000
usertrap(): unexpected scause 0xf pid=7842
            sepc=0x4442 stval=0x33013000
usertrap(): unexpected scause 0xf pid=7843
            sepc=0x4442 stval=0x34013000
usertrap(): unexpected scause 0xf pid=7844
            sepc=0x4442 stval=0x35013000
usertrap(): unexpected scause 0xf pid=7845
            sepc=0x4442 stval=0x36013000
usertrap(): unexpected scause 0xf pid=7846
            sepc=0x4442 stval=0x37013000
usertrap(): unexpected scause 0xf pid=7847
            sepc=0x4442 stval=0x38013000
usertrap(): unexpected scause 0xf pid=7848
            sepc=0x4442 stval=0x39013000
usertrap(): unexpected scause 0xf pid=7849
            sepc=0x4442 stval=0x3a013000
usertrap(): unexpected scause 0xf pid=7850
            sepc=0x4442 stval=0x3b013000
usertrap(): unexpected scause 0xf pid=7851
            sepc=0x4442 stval=0x3c013000
usertrap(): unexpected scause 0xf pid=7852
            sepc=0x4442 stval=0x3d013000
usertrap(): unexpected scause 0xf pid=7853
            sepc=0x4442 stval=0x3e013000
usertrap(): unexpected scause 0xf pid=7854
            sepc=0x4442 stval=0x3f013000
OK
test lazy_copy: OK
ALL TESTS PASSED
$
```

Copy-on-write fork

Design: fork maps the parent's physical pages into the child as read-only with a COW marker, and a write fault does the copy. Each physical page has a reference count, and a page is freed only when its count reaches 0.

Changes (cow branch)

1. kernel/riscv.h: #define PTE_COW (1L << 8) (an RSW bit, set only on pages that were writable at fork).
2. kernel/kalloc.c:
    - struct { struct spinlock lock; int cnt[PHYSTOP/PGSIZE]; } ref; and #define IDX(pa) ((uint64)(pa)/PGSIZE)
    - kinit: initlock(&ref.lock, "ref");
    - freerange: ref.cnt[IDX(p)] = 1; before kfree(p)
    - kfree: decrement the count, and only free the page when it reaches 0
    - kalloc: set ref.cnt[IDX(r)] = 1 for the new page (inside if(r){ ... })
    - new krefinc(void*) and krefcnt(void*)
3. kernel/vm.c:
    - uvmcopy: no copying. For writable PTEs it clears PTE_W and sets PTE_COW in the parent, maps the same pa into the child with the same flags, and calls krefinc. It keeps the continue for unmapped pages.
    - cowalloc(pt, va): returns -1 unless the PTE is valid, user, and COW. If krefcnt == 1, it restores PTE_W and clears PTE_COW. Otherwise it kallocs a page, copies, remaps with PTE_W and without PTE_COW, and kfrees the old page.
    - vmfault: at the top, for write faults on a valid COW page it calls cowalloc and returns nonzero on success.
    - copyout: after computing va0, it checks the PTE and calls cowalloc if it is COW.
4. kernel/defs.h: prototypes for cowalloc, krefinc and krefcnt.
5. kernel/trap.c: no COW changes. The store fault (scause 15) already reaches vmfault, which now handles COW.

Differences from the lab's suggested plan

- A separate cowalloc() helper, called from vmfault.
- The PTE_COW flag distinguishes COW pages from genuinely read-only pages such as text. Writes to those still end in the kill path.
- The refcount==1 shortcut in cowalloc skips a pointless copy for the last owner.

Test: make clean; make qemu, then cowtest (expect ALL COW TESTS PASSED) and usertests -q (expect ALL TESTS PASSED). Both pass.

Build errors we hit and the fixes

Error	Cause	Fix
printf undefined at link time	The util/traps tree uses printk, not printf	Use printk in vm.c on that tree
Stray printf prototypes in defs.h	One was glued onto the either_copyout line	Delete them
%p format error	printk has a format attribute, and pte and pa are uint64	Use %lx
krefinc/krefcnt/cowalloc implicit declaration	Prototypes missing from defs.h	Add them
got_nkernel/vm.cull in copyinstr	Corrupted line (path pasted into got_null)	sed it back to got_null
PTE_COW undeclared	Define missing from riscv.h	Add it after PTE_U
vmfault argument count mismatch	The prototype differs between trees (4 args on util/traps, 3 on cow)	Make the call in trap.c match defs.h
misleading-indentation in kalloc	if(r) had no braces around the added lines	Add { }
"expected declaration or statement at end of input"	Missing closing } in kalloc	Close the if(r){ block and the function
Unused variable pte in copyout	Duplicate declaration	Keep a single pte declaration

Error	Cause	Fix
exec cowtest failed	The traps tree has no cowtest	Work on the cow branch
Checkout blocked by local changes	Uncommitted COW edits on the traps tree	Commit to a scratch branch, then take only the COW hunks
alarm_* members missing in trap.c	Traps-lab alarm code came over to the cow branch	git checkout HEAD -- kernel/trap.c

Branches and commits

- Part 1 belongs on util (or pgtbl). Part 2 belongs on cow.
- Keep them in separate commits so the pgprint changes stay out of the COW diff.
- Part 1:
- git add kernel/vm.c kernel/defs.h kernel/syscall.h kernel/syscall.c kernel/sysproc.c user/usys.pl user/user.h user/sbrktest.c Makefilegit commit -m "Add pgprint syscall and sbrktest to inspect page table around sbrk(1)"
- Part 2:
- git add kernel/riscv.h kernel/kalloc.c kernel/vm.c kernel/defs.hgit commit -m "Implement copy-on-write fork with page refcounts"
- Don't commit build output (fs.img, fs.img.bk, .o files).
- Push with git push -u origin <branch>. Use a private repo unless your instructor allows public solutions.
- Earlier WIP work is on the scratch branch wip-cow. Delete it once the real branches are committed (git branch -D wip-cow).